

GUÍA TÉCNICA DE INSTALACIÓN

Cómo levantar EJB MANAGER

Preparación del entorno, base de datos, ejecución local,
validación y solución de problemas.

Esta guía explica cómo ejecutar EJB MANAGER localmente en Windows. El proyecto está formado por:

- Frontend: React + Vite, puerto 5173 .
- Backend: Node.js + Express, puerto 4000 .
- Base de datos: PostgreSQL 16, puerto 5432 .
- ORM y migraciones: Prisma.
- Videollamadas opcionales: LiveKit, puerto 7880 .

1. Requisitos

Instala antes de comenzar:

- Node.js 20 LTS o una versión posterior compatible.
- pnpm mediante Corepack.
- Docker Desktop con Docker Compose.
- Git, si el proyecto se obtiene desde un repositorio.

Comprueba las instalaciones desde PowerShell:

POWERSHELL

```
node --version
corepack enable
pnpm --version
docker --version
docker compose version
```

Docker Desktop debe estar abierto antes de iniciar PostgreSQL.

2. Abrir el proyecto

Abre PowerShell y entra en la carpeta raíz:

POWERSHELL

```
Set-Location "C:\Proyecto Innovation Hub"
```

Todos los comandos siguientes deben ejecutarse desde esta carpeta, salvo que se indique lo contrario.

3. Configurar las variables de entorno

Crea el archivo local del backend a partir del ejemplo:

POWERSHELL

```
Copy-Item "backend\.env.example" "backend\.env"
```

El contenido mínimo para desarrollo local es:

ENV

```
DATABASE_URL="postgresql://ejb:ejb_local@localhost:5432/ejb_innovation_hub?schema=public"
PORT=4000
FRONTEND_URL=http://localhost:5173
JWT_SECRET="cambia-esta-clave-en-produccion"
LIVEKIT_URL="ws://localhost:7880"
LIVEKIT_API_KEY="devkey"
LIVEKIT_API_SECRET="secret"
ENABLE_HUDDLES=false
```

Consideraciones:

- Para producción, `JWT_SECRET` debe ser aleatorio y tener al menos 32 caracteres.
- Si el frontend utiliza el certificado local incluido en `.certs`, conviene cambiar `FRONTEND_URL` a `https://localhost:5173`.
- Gmail es opcional. Su configuración se encuentra en `outputs/CONFIGURAR_GMAIL.md`.
- Las videollamadas permanecen desactivadas mientras `ENABLE_HUDDLES=false`.

4. Iniciar PostgreSQL

Para levantar únicamente la base de datos:

POWERSHELL

```
docker compose up -d postgres
```

Comprueba que esté funcionando:

POWERSHELL

```
docker compose ps
```

El contenedor crea la base `ejb_innovation_hub` con estas credenciales locales:

Dato	Valor
Servidor	<code>localhost</code>
Puerto	<code>5432</code>
Base de datos	<code>ejb_innovation_hub</code>
Usuario	<code>ejb</code>
Contraseña	<code>ejb_local</code>

Para habilitar también LiveKit:

POWERSHELL

```
docker compose up -d postgres livekit
```

Después cambia `ENABLE_HUDDLES=true` en `backend/.env`.

5. Instalar las dependencias

El repositorio es un workspace de pnpm, por lo que basta con instalar desde la raíz:

POWERSHELL

```
pnpm install
```

Esto instala las dependencias del proyecto raíz, frontend y backend.

6. Preparar la base de datos

Genera el cliente de Prisma y aplica las migraciones:

POWERSHELL

```
pnpm --dir backend prisma:generate  
pnpm --dir backend prisma:migrate
```

Las migraciones crean las tablas y los catálogos necesarios sin borrar los datos existentes.

Advertencia: no ejecutes `pnpm --dir backend prisma:seed` en una base con información. El `seed` actual elimina iniciativas, usuarios, objetivos y áreas antes de recrear el catálogo; está pensado únicamente para reinicializaciones controladas de desarrollo.

7. Iniciar el proyecto

Desde la raíz, ejecuta:

POWERSHELL

```
pnpm dev
```

Este comando levanta frontend y backend simultáneamente. Debes ver mensajes similares a:

TEXT

```
API lista en la red local, puerto 4000  
Local: http://localhost:5173
```

Accesos:

- Aplicación: `http://localhost:5173`
- API: `http://localhost:4000/api`
- Estado del backend: `http://localhost:4000/api/health`

Si existe `.certs/ejb-manager-local.pfx`, Vite iniciará con HTTPS y la aplicación estará en:

TEXT

```
https://localhost:5173
```

Al ser un certificado local, el navegador puede solicitar su aceptación la primera vez.

8. Comprobar que todo funciona

En otra ventana de PowerShell, consulta el estado de la API:

POWERSHELL

```
Invoke-RestMethod "http://localhost:4000/api/health"
```

La respuesta esperada es:

TEXT

```
status  
-----  
ok
```

Después abre la aplicación, registra o inicia sesión con un usuario y confirma que el Resumen cargue información sin mostrar errores de conexión.

Inicio rápido para los siguientes días

Una vez realizada la instalación inicial, normalmente solo necesitas:

POWERSHELL

```
Set-Location "C:\Proyecto Innovation Hub"  
docker compose up -d postgres  
pnpm dev
```

Después de recibir cambios del proyecto

Cuando se actualice el código, ejecuta:

POWERSHELL

```
pnpm install  
pnpm --dir backend prisma:generate  
pnpm --dir backend prisma:migrate  
pnpm dev
```

Compilar y validar

Para comprobar frontend y backend:

POWERSHELL

```
pnpm build  
pnpm --dir backend test
```

Para verificar la configuración mínima de producción:

POWERSHELL

```
pnpm --dir backend production:check
```

Ejecutar una compilación local

Primero genera los archivos de producción:

POWERSHELL

```
pnpm build
```

Luego abre dos terminales desde la raíz.

Terminal 1, backend:

POWERSHELL

```
pnpm --dir backend start
```

Terminal 2, frontend:

POWERSHELL

```
pnpm --dir frontend preview
```

Detener el proyecto

- Presiona `Ctrl + C` en la terminal que ejecuta `pnpm dev`.
- Para detener los servicios de Docker:

POWERSHELL

```
docker compose stop
```

Para retirar los contenedores sin eliminar la base persistida:

POWERSHELL

```
docker compose down
```

No uses `docker compose down -v` salvo que realmente quieras eliminar todo el volumen y los datos de PostgreSQL.

Errores frecuentes

Docker no está disponible

Síntoma: el comando `docker compose up` no conecta con el motor.

Solución: abre Docker Desktop, espera a que termine de iniciar y vuelve a ejecutar el comando.

Prisma muestra `P1001`

El backend no puede conectarse a PostgreSQL. Comprueba:

POWERSHELL

```
docker compose ps
docker compose logs postgres
```

También revisa que `DATABASE_URL` en `backend/.env` coincida con las credenciales de `docker-compose.yml`.

El puerto ya está ocupado

Comprueba los puertos utilizados:

POWERSHELL

```
Get-NetTCPConnection -LocalPort 4000,5173,5432 -ErrorAction SilentlyContinue
```

Cierra el proceso anterior o modifica el puerto correspondiente antes de reiniciar.

El frontend abre, pero no carga datos

Verifica primero la API:

POWERSHELL

```
Invoke-RestMethod "http://localhost:4000/api/health"
```

Si no responde, revisa la terminal del backend y los logs de PostgreSQL.

El navegador advierte sobre el certificado

La aplicación detectó el certificado autofirmado de `.certs`. Puedes aceptar el certificado local o retirar temporalmente ese archivo para que Vite utilice HTTP durante el desarrollo.

pnpm no se reconoce

Ejecuta:

POWERSHELL

```
corepack enable  
corepack prepare pnpm@latest --activate
```

Cierra y vuelve a abrir PowerShell si el comando todavía no aparece.

Resumen de comandos

POWERSHELL

```
Set-Location "C:\Proyecto Innovation Hub"  
Copy-Item "backend\.env.example" "backend\.env"  
docker compose up -d postgres  
pnpm install  
pnpm --dir backend prisma:generate  
pnpm --dir backend prisma:migrate  
pnpm dev
```